

FRD

Functional Requirements Document (FRD)

AWS Glue Data Pipeline for Customer Order Processing

-

1. Document Overview

Purpose

This Functional Requirements Document defines the functional specifications for an AWS Glue-based data pipeline that ingests customer and order data from S3, performs data quality operations, implements SCD Type 2 logic using Apache Hudi, and produces analytical aggregations. The pipeline will use AWS Glue for ETL processing, Glue Data Catalog for metadata management, and S3 for storage.

Scope

- In Scope:
 - Ingestion of customer and order CSV files from specified S3 locations
 - Data cleaning operations (null removal, deduplication)
 - Registration of datasets in AWS Glue Data Catalog
 - Implementation of SCD Type 2 logic for order summary using Apache Hudi
 - Incremental processing triggered by customer data changes
 - Generation of customer aggregate spend analytics table
 - Integration with AWS services: S3, Glue, Athena, CloudWatch
- Out of Scope:
 - Real-time streaming data processing
 - Data encryption and security policies (assumed to be handled by infrastructure)
 - Lake Formation governance implementation (optional component)
 - Data archival and retention policies
 - User interface or reporting layer

Assumptions

1. CSV files in source S3 locations follow consistent schema and naming conventions
2. Source data arrives in complete files (not partial uploads)
3. AWS Glue job has appropriate IAM permissions to read from source buckets and write to target buckets
4. Glue Data Catalog database `gen\_ai\_poc\_databrickscoe` already exists
5. S3 bucket paths are accessible and have appropriate lifecycle policies configured

6. "Null" (string) and NULL (actual null) are the only null representations requiring removal
 7. Duplicate detection is based on all columns in each dataset
 8. Customer updates are identified by changes in any customer attribute for a given CustId
 9. Date fields in source data are in a parseable format
 10. CloudWatch logging is enabled for Glue jobs
 11. Athena is used for querying cataloged tables but query patterns are not defined in this scope
-

2. Functional Requirements

FR-INGEST-001: Ingest Customer Data from S3

- Title: Load Customer CSV Files from S3 Source Location
- Description:

The system shall read customer data in CSV format from the designated S3 path and load it into the processing pipeline.

- Business Objective:

Enable the pipeline to access current customer information for downstream processing and analytics.

- Inputs:
 - Source: s3://adif-sdlc/sdlc_wizard/customerdata/
 - Format: CSV
 - Schema:
 - CustId: (datatype not specified in BRD)
 - Name: (datatype not specified in BRD)
 - EmailId: (datatype not specified in BRD)
 - Region: (datatype not specified in BRD)
- Processing Rules / Functional Steps:
 1. Connect to S3 source path s3://adif-sdlc/sdlc_wizard/customerdata/
 2. Read all CSV files matching the customer data pattern
 3. Parse CSV with header row
 4. Load data into Glue DynamicFrame or Spark DataFrame
- Outputs:
 - In-memory customer dataset ready for cleaning operations
- Preconditions:

- S3 bucket and path exist and are accessible
- CSV files are present in the source location
- Glue job has read permissions on the S3 path
- Postconditions:
- Customer data is loaded into memory for processing
- No data is written to persistent storage at this stage
- Error / Validation Conditions:
- If S3 path does not exist, job should fail with clear error message
- If no CSV files are found, job should fail or log warning based on configuration
- If CSV structure does not match expected schema, job should fail with schema mismatch error
-

FR-INGEST-002: Ingest Order Data from S3

- Title: Load Order CSV Files from S3 Source Location
- Description:

The system shall read order data in CSV format from the designated S3 path and load it into the processing pipeline.

- Business Objective:

Enable the pipeline to access transactional order information for joining with customer data and generating analytics.

- Inputs:
- Source: s3://adif-sdlc/sdlc_wizard/orderdata/
- Format: CSV
- Schema:
- OrderId: (datatype not specified in BRD)
- ItemName: (datatype not specified in BRD)
- PricePerUnit: (datatype not specified in BRD)
- Qty: (datatype not specified in BRD)
- Date: (datatype not specified in BRD)
- CustId: (datatype not specified in BRD)
- Processing Rules / Functional Steps:
- 1. Connect to S3 source path s3://adif-sdlc/sdlc_wizard/orderdata/
- 2. Read all CSV files matching the order data pattern

3. Parse CSV with header row

4. Load data into Glue DynamicFrame or Spark DataFrame

- Outputs:

- In-memory order dataset ready for cleaning operations

- Preconditions:

- S3 bucket and path exist and are accessible

- CSV files are present in the source location

- Glue job has read permissions on the S3 path

- Postconditions:

- Order data is loaded into memory for processing

- No data is written to persistent storage at this stage

- Error / Validation Conditions:

- If S3 path does not exist, job should fail with clear error message

- If no CSV files are found, job should fail or log warning based on configuration

- If CSV structure does not match expected schema, job should fail with schema mismatch error

-

FR-CLEAN-001: Remove Null Values from Customer and Order Datasets

- Title: Data Quality - Null Value Removal

- Description:

The system shall remove all records containing "Null" (string literal) or NULL (actual null values) from both customer and order datasets.

- Business Objective:

Ensure data quality by eliminating incomplete records that could cause downstream processing errors or inaccurate analytics.

- Inputs:

- In-memory customer dataset from FR-INGEST-001

- In-memory order dataset from FR-INGEST-002

- Processing Rules / Functional Steps:

1. For customer dataset:

- Identify rows where any column contains the string "Null" (case-sensitive)

- Identify rows where any column contains NULL (actual null value)

- Remove all identified rows

2. For order dataset:

- Identify rows where any column contains the string "Null" (case-sensitive)
- Identify rows where any column contains NULL (actual null value)
- Remove all identified rows

3. Log count of removed records for each dataset

• Outputs:

- Cleaned customer dataset with no null values
- Cleaned order dataset with no null values
- Log entries showing number of records removed from each dataset

• Preconditions:

- Customer and order datasets are loaded in memory

• Postconditions:

- All remaining records have non-null values in all columns
- Datasets are ready for duplicate removal

• Error / Validation Conditions:

- If all records are removed due to nulls, log warning but continue processing
- Processing should not fail if no nulls are found

•

FR-CLEAN-002: Remove Duplicate Records from Customer and Order Datasets

- Title: Data Quality - Duplicate Record Removal

• Description:

The system shall identify and remove duplicate records from both customer and order datasets based on all column values.

• Business Objective:

Prevent duplicate data from causing incorrect aggregations, inflated metrics, or SCD Type 2 logic errors.

• Inputs:

- Null-cleaned customer dataset from FR-CLEAN-001
- Null-cleaned order dataset from FR-CLEAN-001

• Processing Rules / Functional Steps:

1. For customer dataset:

- Identify duplicate rows where all columns (CustId, Name, EmailId, Region) match
- Retain only one instance of each duplicate set
- Remove all other duplicate instances

2. For order dataset:

- Identify duplicate rows where all columns (OrderId, ItemName, PricePerUnit, Qty, Date, CustId) match
- Retain only one instance of each duplicate set
- Remove all other duplicate instances

3. Log count of removed duplicates for each dataset

- Outputs:
 - Deduplicated customer dataset
 - Deduplicated order dataset
 - Log entries showing number of duplicates removed from each dataset
- Preconditions:
 - Customer and order datasets have been null-cleaned
- Postconditions:
 - Each unique record appears exactly once in each dataset
 - Datasets are ready for catalog registration
- Error / Validation Conditions:
 - If all records are duplicates (leaving zero records), log error and fail job
 - Processing should not fail if no duplicates are found
-

FR-CATALOG-001: Register Customer Table in Glue Data Catalog

- Title: Catalog Customer Data as Glue Table
- Description:

The system shall register the cleaned customer dataset as a table in the AWS Glue Data Catalog for query access via Athena.

- Business Objective:

Make customer data discoverable and queryable through standard SQL interfaces for analytics and validation purposes.

- Inputs:
 - Deduplicated customer dataset from FR-CLEAN-002

- Processing Rules / Functional Steps:

1. Write cleaned customer data to S3 in a queryable format (Parquet recommended)
2. Register table in Glue Data Catalog with the following specifications:

- Database: gen_ai_poc_databricksco
- Table name: sdlc_wizard_customer
- Schema: CustId, Name, EmailId, Region

3. Update table metadata and statistics

- Outputs:

- Customer data persisted in S3
- Glue Catalog table `gen\_ai\_poc\_databricksco.sdlc\_wizard\_customer` created/updated
- Table queryable via Athena

- Preconditions:

- Glue Data Catalog database `gen\_ai\_poc\_databricksco` exists
- Glue job has permissions to write to catalog
- Customer data has been cleaned

- Postconditions:

- Customer table is registered and queryable
- Table metadata reflects current schema

- Error / Validation Conditions:

- If database does not exist, job should fail with clear error
- If table registration fails, job should fail and rollback S3 writes if possible
- If S3 write fails, job should fail before catalog registration

-

FR-CATALOG-002: Register Order Table in Glue Data Catalog

- Title: Catalog Order Data as Glue Table

- Description:

The system shall register the cleaned order dataset as a table in the AWS Glue Data Catalog for query access via Athena.

- Business Objective:

Make order data discoverable and queryable through standard SQL interfaces for analytics and validation purposes.

- Inputs:
- Deduplicated order dataset from FR-CLEAN-002
- Processing Rules / Functional Steps:
 1. Write cleaned order data to S3 in a queryable format (Parquet recommended)
 2. Register table in Glue Data Catalog with the following specifications:
 - Database: gen_ai_poc_databricksco
 - Table name: sdlc_wizard_order
 - Schema: OrderId, ItemName, PricePerUnit, Qty, Date, CustId
 3. Update table metadata and statistics
- Outputs:
- Order data persisted in S3
- Glue Catalog table `gen\_ai\_poc\_databricksco.sdlc\_wizard\_order` created/updated
- Table queryable via Athena
- Preconditions:
 - Glue Data Catalog database `gen\_ai\_poc\_databricksco` exists
 - Glue job has permissions to write to catalog
 - Order data has been cleaned
- Postconditions:
 - Order table is registered and queryable
 - Table metadata reflects current schema
- Error / Validation Conditions:
 - If database does not exist, job should fail with clear error
 - If table registration fails, job should fail and rollback S3 writes if possible
 - If S3 write fails, job should fail before catalog registration
-

FR-SCD2-001: Implement SCD Type 2 Logic for Order Summary

- Title: Slowly Changing Dimension Type 2 for Order Summary Table
- Description:

The system shall implement SCD Type 2 logic to maintain historical changes in the order summary table using Apache Hudi format on S3. When customer or order data changes, the system shall close existing active records and insert new active records with updated information.

- Business Objective:

Maintain complete historical tracking of customer order information, enabling time-based analytics and audit trails while preserving the ability to query current and historical states.

- Inputs:

- Cleaned customer dataset from FR-CATALOG-001
- Cleaned order dataset from FR-CATALOG-002
- Existing ordersummary table (if present) from s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

- Processing Rules / Functional Steps:

1. Join customer and order datasets on CustId
2. For each joined record, compare against existing active records in ordersummary table
3. If record is new (no matching key in ordersummary):
 - Insert new record with IsActive = true, StartDate = current timestamp, EndDate = null, OpTs = current timestamp
4. If record exists and has changes in any attribute:
 - Update existing active record: set IsActive = false, EndDate = current timestamp, OpTs = current timestamp
 - Insert new record with updated values: IsActive = true, StartDate = current timestamp, EndDate = null, OpTs = current timestamp
5. If record exists and has no changes:
 - No action required
6. Write results to S3 using Hudi format with upsert operation
7. Update Glue Catalog table metadata

- Outputs:

- Updated ordersummary table at s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Glue Catalog table `gen\_ai\_poc\_databrickscoe.ordersummary` updated
- SCD Type 2 columns maintained: IsActive, StartDate, EndDate, OpTs

- Preconditions:

- Customer and order data are cleaned and cataloged
- S3 target path is accessible
- Hudi libraries are available in Glue environment
- Glue job has write permissions to target S3 path

- Postconditions:

- All current records have IsActive = true and EndDate = null
- All historical records have IsActive = false and EndDate populated

- Complete audit trail of changes is maintained
- Table is queryable via Athena with time-travel capabilities
- Error / Validation Conditions:
 - If join produces no records, log warning but do not fail
 - If Hudi write fails, job should fail and not update catalog
 - If multiple active records exist for same key after processing, job should fail with data integrity error
 - If StartDate > EndDate for any record, job should fail with validation error
-

FR-SCD2-002: Incremental Processing Based on Customer Updates

- Title: Trigger SCD Type 2 Updates on Customer Data Changes
- Description:

The system shall detect changes in customer data and trigger corresponding SCD Type 2 updates in the ordersummary table, ensuring that customer attribute changes are reflected in the historical order summary.

- Business Objective:

Maintain data consistency between customer master data and order summary, ensuring that customer information changes are properly historized in downstream analytical tables.

- Inputs:
 - New/updated customer data from s3://adif-sdlc/sdlc_wizard/customerdata/
 - Existing customer data from previous run (tracked via Glue job bookmarks or comparison logic)
- Processing Rules / Functional Steps:
 1. Compare incoming customer data against previously processed customer data
 2. Identify customer records with changes in CustId, Name, EmailId, or Region
 3. For each changed customer:
 - Retrieve all related order records from order dataset
 - Trigger SCD Type 2 update logic (FR-SCD2-001) for affected order summary records
 4. Process only changed customers to optimize performance (incremental processing)
 5. Update job bookmark or tracking mechanism to record processed customer state
- Outputs:
 - Updated ordersummary table reflecting customer changes
 - Job bookmark or state file indicating last processed customer data state

- Preconditions:
- Customer data ingestion (FR-INGEST-001) has completed
- Order data ingestion (FR-INGEST-002) has completed
- Ordersummary table exists (may be empty on first run)
- Postconditions:
- All customer changes are reflected in ordersummary with proper SCD Type 2 history
- Job state is updated for next incremental run
- Error / Validation Conditions:
- If unable to determine previous customer state on first run, process all customers as new
- If customer change detection fails, fall back to full refresh mode
- If no customer changes detected, skip SCD Type 2 processing and log informational message
-

FR-AGG-001: Generate Customer Aggregate Spend Table

- Title: Create Customer Aggregate Spend Analytics Table
- Description:

The system shall generate an aggregated analytics table that summarizes total spending per customer, providing a denormalized view for reporting and analytics purposes.

- Business Objective:

Enable fast analytical queries on customer spending patterns without requiring complex joins or aggregations at query time.

- Inputs:
- Ordersummary table from s3://adif-sdlc/curated/sdlc_wizard/ordersummary/ (active records only)
- Processing Rules / Functional Steps:
- 1. Read active records from ordersummary table (where IsActive = true)
- 2. Calculate TotalAmount for each order: $\text{PricePerUnit} \times \text{Qty}$
- 3. Group by customer Name and Date
- 4. Aggregate: $\text{sum}(\text{TotalAmount})$ as TotalAmount
- 5. Select columns: Name, TotalAmount, Date
- 6. Write results to S3 location: s3://adif-sdlc/analytics/customeraggregatespend/
- 7. Register/update table in Glue Data Catalog:
- Database: gen_ai_poc_databrickscoe
- Table name: customeraggregatespend

- Schema: Name, TotalAmount, Date
- Outputs:
 - Customer aggregate spend data at s3://adif-sdlc/analytics/customeragggregatespend/
 - Glue Catalog table `gen\_ai\_poc\_databrickscoe.customeragggregatespend` created/updated
 - Table queryable via Athena
- Preconditions:
 - Ordersummary table exists and contains data
 - SCD Type 2 processing (FR-SCD2-001) has completed
 - S3 target path is accessible
- Postconditions:
 - Aggregate table contains one row per customer per date with total spending
 - Table is optimized for analytical queries
 - Table reflects only current active order summary records
- Error / Validation Conditions:
 - If ordersummary table is empty, create empty customeragggregatespend table
 - If TotalAmount calculation results in negative values, log warning but include in output
 - If aggregation produces no results, log warning but do not fail job
 - If S3 write fails, job should fail before catalog registration
-

3. Non-Functional Considerations

Performance

- Pipeline should leverage Glue's distributed Spark processing for scalability
- Hudi upsert operations should be optimized with appropriate partitioning strategy
- Incremental processing (FR-SCD2-002) should minimize full table scans

Monitoring and Observability

- All Glue jobs should log to CloudWatch
- Job metrics (records processed, duration, errors) should be captured
- CloudWatch alarms should be configurable for job failures

Data Quality

- Data cleaning operations (FR-CLEAN-001, FR-CLEAN-002) should log statistics
- Validation checks should be performed at each stage

- Data quality metrics should be published to CloudWatch

Availability

- Pipeline should be idempotent to support reruns
- Failed jobs should be retryable without data corruption
- Glue job bookmarks should enable incremental processing

Governance (Optional)

- Lake Formation can be integrated for fine-grained access control
- Data lineage should be trackable through Glue Data Catalog
- Table-level and column-level permissions can be managed via Lake Formation
-

4. Requirement Traceability Notes

Requirement ID	BRD Section	Key Traceability Points	
-----	-----	-----	
FR-INGEST-001	Source Input (1), Schemas (2)	Customer CSV source path, schema definition	
FR-INGEST-002	Source Input (1), Schemas (2)	Order CSV source path, schema definition	
FR-CLEAN-001	Data Cleaning (3)	Null removal requirement	
FR-CLEAN-002	Data Cleaning (3)	Duplicate removal requirement	
FR-CATALOG-001	Glue Catalog Tables (4)	Customer table registration	
FR-CATALOG-002	Glue Catalog Tables (4)	Order table registration	
FR-SCD2-001	SCD Type 2 (5)	Hudi implementation, ordersummary table, SCD2 logic	
FR-SCD2-002	Incremental Logic (6)	Customer update triggers, incremental processing	
FR-AGG-001	Aggregation Table (7)	customeraggregatespend table, aggregation logic	

- Cross-Requirement Dependencies:
- FR-CATALOG-001 and FR-CATALOG-002 depend on FR-CLEAN-001 and FR-CLEAN-002
- FR-SCD2-001 depends on FR-CATALOG-001 and FR-CATALOG-002
- FR-SCD2-002 depends on FR-SCD2-001
- FR-AGG-001 depends on FR-SCD2-001

- AWS Architecture Alignment:

All functional requirements align with the AWS components specified in BRD section 8:

- Storage: S3 (all FR requirements)
- ETL: AWS Glue with Spark and Hudi (all processing requirements)
- Catalog: Glue Data Catalog (FR-CATALOG-001, FR-CATALOG-002, FR-SCD2-001, FR-AGG-001)

- Query: Athena (implicit in all catalog requirements)
- Monitoring: CloudWatch (referenced in non-functional considerations)
-
- Document Version: 1.0
- Last Updated: 2025-01-10
- Status: Ready for Technical Requirements Document (TRD) Development